

Uygulama Programlama Arayüzleri (API) ve Modern Web Mimarisi

NTP II – Hafta 8



Uygulama Programlama Arayüzleri (API) ve Modern Web Mimarisi

Bu sunum serisi boyunca modern web uygulamalarının temel taşı olan API kavramını öğrenecek, yüksek performanslı Python çatısı FastAPI'yi inceleyeceğiz.
Kurulum süreçleri ve profesyonel hata ayıklama teknikleri (debugging) odak noktamız olacaktır.



Modül Öğrenme Hedefleri

API (Application Programming Interface) kavramını ve modern web'deki rolünü anlamak.

Neden FastAPI'nin tercih edildiğini (Performans ve otomatik dokümantasyon) kavramak.

Profesyonel bir IDE olarak PyCharm'ı kurmak ve yapılandırmak.

Sanal Ortamların ('kütüphane cehennemini' çözme) önemini anlamak.

uvicorn ile sunucuyu başlatmak ve ilk 'Hello, World!' endpoint'ini test etmek.

PyCharm debugger'ı kullanarak kod akışını izlemeyi öğrenmek.

API Nedir? Temel Tanım

API, iki yazılım bileşeninin birbiriyle iletişim kurmasını sağlayan bir dizi kural ve protokoller bütünüdür.

Basitçe, bir yazılımın diğerine ne yapmasını isteyebileceğini ve yanıt olarak ne bekleyebileceğini belirleyen bir araçtır (contract).



API'lerin Modern Web'deki Rolü

Günümüzün karmaşık uygulamaları (Instagram, Spotify) monolitik yapılar yerine mikroservis mimarisini kullanır.

API'ler, farklı servislerin (örneğin, kullanıcı doğrulama servisi, ödeme servisi, içerik servisi) birbirinden bağımsız çalışmasını ve veri alışverişi yapmasını sağlar.

Bu, ölçeklenebilirlik ve geliştirme hızını artırır.



RESTful API Mimarisi

REST, API'ler için en yaygın mimari stildir. Durum bilgisiz (stateless) ve kaynak tabanlıdır (resource-based).

Temel işlemler HTTP metotları (GET, POST, PUT, DELETE) kullanılarak gerçekleştirilir.

FastAPI, doğal olarak RESTful prensiplere uygun API'ler oluşturmayı hedefler.

API Kullanım Senaryosu: Tehdit İstihbaratı (Threat Intel) API'si

Senaryo: Bir Geliştirme Ekibinin İlk Görevi.

Talep: Sunucunun ayakta ve erişilebilir olduğunu doğrulamak için basit bir GET /health endpoint'ine ihtiyacımız var.

Yanıt Beklentisi: {'status': 'ok'} şeklinde bir JSON yanıtı dönmeli.

Çözüm: Bu hafta, bu temel 'sağlık kontrolü' endpoint'ini oluşturarak sistemin temelini atacağız.



Neden FastAPI? Yüksek Performans

Geleneksel Python web çatılarının çoğu (Flask, Django) WSGI (Web Server Gateway Interface) standardını kullanır.

FastAPI, asenkron işlemleri destekleyen ASGI (Asynchronous Server Gateway Interface) üzerine inşa edilmiştir.

ASGI sayesinde FastAPI, eşzamanlı (concurrent) işlemleri çok daha verimli yönetir, bu da ona Flask gibi WSGI tabanlı araçlardan kat kat daha fazla performans sağlar.



ASGI: Python'da Asenkronluğun Gücü

ASGI, modern Python asenkron kütüphaneleri (async/await) ile uyumludur. Bu, I/O yoğun görevlerde (veritabanı sorguları, harici API çağrıları) uygulamanın bir isteği beklerken diğer isteği işleyebilmesi anlamına gelir. FastAPI, bu asenkron yetenekleri Uvicorn gibi ASGI sunucularıyla birleştirerek sıfır engelleme (non-blocking) operasyon sağlar.



Neden FastAPI? Otomatik Dokümantasyon

FastAPI'nin en büyük avantajlarından biri, API uç noktalarını tanımladığınız an otomatik olarak etkileşimli dokümantasyon oluşturmaktır. Bu dokümantasyon, OpenAPI (eski adıyla Swagger) standardına dayanır. Geliştiriciler, /docs (Swagger UI) ve /redoc adreslerine giderek API'yi test edebilir ve yapısını görebilirler. Bu, özellikle büyük ekipler için zaman tasarrufu sağlar.

Pydantic ile Veri Doğrulama

FastAPI, Python'ın tip ipuçlarını (type hinting) kullanarak Pydantic kütüphanesi aracılığıyla veri modellerini tanımlar.

Bu, gelen isteklerin (JSON payload) otomatik olarak doğrulanmasını (validation) ve serileştirilmesini (serialization) sağlar.

Yanlış veri tipleri veya eksik alanlar otomatik olarak 422 Unprocessable Entity hatası döndürür, bu da API güvenliğini artırır.



Profesyonel Geliştirme Ortamının Önemi

Kod editörleri (VS Code, Sublime) metin yazımı için yeterli olsa da, IDE'ler (Entegre Geliştirme Ortamı) daha fazlasını sunar.

PyCharm gibi bir IDE, otomatik tamamlama, kod refactoring, versiyon kontrolü entegrasyonu ve en önemlisi görsel hata ayıklama (debugger) gibi özelliklere sahiptir.

Bu araçlar, karmaşık projelerde verimliliği ve hata yakalama yeteneğini artırır.



PyCharm Kurulumu ve Özellikleri

PyCharm, JetBrains tarafından geliştirilen popüler bir Python IDE'sidir. Kurulum aşamasında, Community (ücretsiz) veya Professional (FastAPI projeleri için daha gelişmiş özelliklere sahip) sürümünü seçebiliriz. Kurulumun ardından temel ayarları (tema, Python yorumlayıcısı) yapılandırmak önemlidir.

Ön Koşullar: Python ve Git Kurulumu

Bu projeye başlamadan önce sisteminizde Python 3.8+ kurulu olmalıdır. Versiyon kontrol sistemi olan Git'in kurulu olması, kodu takip etmek ve ekiplerle çalışmak için zorunludur. Bu kurulumlar, PyCharm'ın projenin ilk aşamalarında otomatik olarak tanıyacağı temel araçlardır.



Kütüphane Cehennemi (Dependency Hell) Sorunu

Farklı Python projeleri, aynı kütüphanenin farklı versiyonlarına ihtiyaç duyabilir. Örneğin, Proje A 'requests==2.0' isterken, Proje B 'requests==3.0' isteyebilir. Bu çakışmalar, 'kütüphane cehennemi' olarak bilinen yaygın bir sorundur ve projelerin yerel makinede çalışmasını imkansız hale getirebilir.

Sanal Ortamların Çözümü: venv

Sanal Ortamlar (venv), her projenin kendi izole edilmiş bağımlılık setine sahip olmasını sağlar.

Bu sayede, bir projenin gereksinimleri diğer projelerle veya sistem genelindeki Python kurulumuyla çakışmaz.

Python -m venv venv komutuyla projenin kök dizininde izole bir dizin oluşturulur.



PyCharm'da Yeni Proje Kurulumu

PyCharm açıldıktan sonra yeni bir proje oluşturulur. Burada ya boş bir proje (Empty Project) ya da doğrudan bir FastAPI proje şablonu (FastAPI Project Template) seçeneği kullanılabilir. Profesyonel projelerde, kaynak kodun izole edilmesi için her zaman sanal ortam (venv) kullanılması şiddetle önerilir.



Sanal Ortamın Oluşturulması ve Aktifleştirilmesi

PyCharm'in dahili terminalini kullanarak sanal ortam oluşturulur: `python -m venv venv`.

Sanal ortamın aktifleştirilmesi: `source venv/bin/activate` (Linux/macOS) veya `.venv Scripts activate` (Windows PowerShell).

Ortam aktifleştirildiğinde, terminal satırının başında (venv) ibaresi görünmelidir.

FastAPI ve Uvicorn Kurulumu

Sanal ortam aktifleştirildikten sonra gerekli kütüphaneler yüklenir.
Temel olarak FastAPI çatısı ve Uvicorn ASGI sunucusu gereklidir.
Komut: `pip install fastapi uvicorn`.
Alternatif olarak: `pip install 'fastapi[all]'` komutu, Uvicorn, Pydantic ve diğer temel bağımlılıkları tek seferde yükler.

Temel Kod Yapısı: main.py

Projenin temelini oluşturacak olan main.py dosyasını oluşturuyoruz. Bu dosya, FastAPI uygulamasının ana tanımını ve tüm endpoint'leri (uç noktaları) içerecektir. İlk satır: `from fastapi import FastAPI`, FastAPI Sınıfını (Class) import ediyoruz.

OOP Bağlantısı: Sınıf ve Nesne Kavramları

`app = FastAPI()`: Bu satır, Nesne Tabanlı Programlama (OOP) dersinin temelini oluşturur.

`FastAPI`, bir 'Sınıf' (Class) iken, `app` ise bu sınıftan yaratılan bir 'Nesne'dir (Object) (instantiation).

Uygulamanın çalışması ve metotlarının çağırılması bu `app` nesnesi üzerinden gerçekleşir.

main.py Kod 1/3: Uygulama Nesnesini Yaratma

```
from fastapi import FastAPI
```

```
# 1. OOP Dersi: FastAPI 'sınıfından' bir 'nesne' (app) yaratıyoruz.  
app = FastAPI()
```

Decorator Kavramı

Decorator'lar (@), bir fonksiyonu veya metodu sarmalayarak davranışını değiştiren Python yapılarıdır.

FastAPI'de, `@app.get('/path')` yapısı, hemen altındaki Python fonksiyonunun belirli bir HTTP GET isteği için çağırılması gerektiğini belirtir.

Bu, app nesnesinin get metodunu bir decorator aracılığıyla çağırmanın kısa yoludur.

main.py Kod 2/3: Decorator Uygulaması

Müşteri talebini karşılamak için /health adresine yönelik bir GET metodu tanımlıyoruz:

2. Decorator: Bu, app nesnesinin 'get' metodunu çağırır.

@app.get('/health')


```
def read_health():
```

```
# 3. FastAPI, bu Python sözlüğünü otomatik olarak JSON'a çevirecek.
```

```
return {'status': 'ok'}
```

FastAPI, Python sözlükleri (dictionary) ve Pydantic modelleri gibi standart Python veri yapılarını otomatik olarak JSON formatına dönüştürme yeteneğine sahiptir.

Özet Kod: İlk Çalışan API

```
from fastapi import FastAPI
```

```
app = FastAPI()
```

```
@app.get('/health')  
def read_health():  
    return {'status': 'ok'}
```

Uvicorn ile Sunucuyu Başlatma

Uygulamamızı çalıştırmak için Uvicorn ASGI sunucusunu kullanırız.

Komut: `uvicorn main:app --reload`.

'main': Kodun bulunduğu Python dosyasının adı (main.py).

'app': main.py dosyasındaki FastAPI nesnesinin adı.

'--reload': Geliştirme sırasında kod değişikliklerini otomatik olarak algılayıp sunucuyu yeniden başlatır.

Sunucu Yanıtını Test Etme

Sunucu başlatıldıktan sonra tarayıcıyı açın ve <http://127.0.0.1:8000/health> adresine gidin.

Beklenen Yanıt: Tarayıcı ekranında `{'status': 'ok'}` JSON yanıtının görünmesi gerekir.

Bu, API'mizin başarıyla konuşlandırıldığını ve temel müşteri talebini karşıladığını gösterir.

Otomatik Dokümantasyona Giriş: Swagger UI

FastAPI'nin en heyecan verici kısımlarından biri otomatik dokümantasyondur. Tarayıcıda <http://127.0.0.1:8000/docs> adresine gidin. Burada, tanımladığınız /health endpoint'inin listelendiği, etkileşimli bir arayüz (Swagger UI) göreceksiniz. Bu arayüzden doğrudan API istekleri yapabilir ve yanıtları inceleyebilirsiniz.

Swagger UI'nin Rolü

Swagger UI, manuel dokümantasyon ihtiyacını ortadan kaldırır ve her zaman güncel kalır.

Ekip üyeleri ve API tüketicileri, kodu okumadan hangi uç noktaların mevcut olduğunu, hangi parametreleri kabul ettiğini ve hangi yanıt tiplerini döndürdüğünü hızlıca anlayabilir.

Bu, Geliştirici Deneyimi (Developer Experience - DX) açısından kritik bir avantajdır.



Profesyonel Beceriler: Debugging'in Önemi

Hata ayıklama (Debugging), bir uygulamanın neden beklenildiği gibi çalışmadığını anlamak için hayati bir beceridir.

PyCharm'ın görsel debugger'ı, kod akışını satır satır izlemeyi (step through) sağlayarak programın çalışma zamanındaki durumunu görmemize olanak tanır.

Bu beceri, sonraki haftalardaki karmaşık hataları çözmek için temel oluşturur.

Breakpoint (Kesme Noktası) Nedir?

Breakpoint, bir programın yürütülmesini belirli bir satırda geçici olarak durduran bir işaretleyicidir.

PyCharm'da bir satır numarasının yanına tıklayarak kırmızı bir nokta koymak, bir breakpoint oluşturur.

Kod bu noktaya ulaştığında durur ve programın o anki durumunu incelememize izin verir.

PyCharm Debug Modunda Başlatma

Uygulamayı normal çalıştırmak yerine, 'Debug' modunda başlatmalıyız. PyCharm'da 'Run' menüsü altındaki 'Debug 'main"' seçeneği ile uygulama debug modunda başlar. Uvicorn sunucusu, artık debugger tarafından kontrol edilen özel bir süreçte çalışır.



Debug Adımı 1: Breakpoint Konumu

`return {'status': 'ok'}` satırına bir breakpoint koyun.
Sunucu debug modunda çalışırken, tarayıcıdan /health adresine istek yapın.
Debugger, kod akışının tam bu satırda durduğunu gösterecektir.



Debug Adımı 2: Değişkenleri İnceleme

Kod durduğunda, PyCharm arayüzündeki 'Variables' (Değişkenler) panelini inceleyin.

Bu panel, programın o anki durumundaki tüm yerel ve global değişkenlerin değerlerini gösterir.

Bu, özellikle daha karmaşık uygulamalarda, bir fonksiyonun neden yanlış bir değer döndürdüğünü anlamak için kritik öneme sahiptir.



Debug Kontrolleri

Debugger araç çubuğundaki temel kontroller:

Step Over (F8): Bir sonraki satıra atlar, fonksiyon çağrılarını içine girmez.

Step Into (F7): Fonksiyon çağrısının içine girerek detaylı inceleme yapar.

Resume Program (F9): Bir sonraki breakpoint'e kadar programın çalışmaya devam etmesini sağlar.



OOP Kavramının Derinleştirilmesi

FastAPI nesnesi (app) yaratmak, bu dersin 'Nesne Tabanlı Programlama' adını meşrulaştırır ve teoriyi pratiğe bağlar.

Bu, öğrencilerin sadece bir kütüphane kullanmadığını, aynı zamanda kalıtım, metotlar ve nesneler gibi temel OOP prensiplerini uyguladığını gösterir.



FastAPI'deki Metotlar ve Uç Noktalar

`app.get`, `app.post`, `app.put`, `app.delete` gibi yapılar, `app` nesnesinin metotlarıdır.

Bu metotlar, belirli HTTP fiillerini (verbs) o nesneye bağlı olarak tanımlamamızı sağlar.

Her bir metot çağrısı, API'mizin dış dünya ile iletişim kuracağı yeni bir kapıyı (endpoint) açar.

Geliştirme Ortamı Seçimi Etkisi

PyCharm gibi güçlü bir IDE kullanmak, hata ayıklama yetenekleri sayesinde karmaşık kod hatalarını bulma süresini yüzde 50'ye kadar azaltabilir. IDE, kod kalitesi denetimleri (linting) ve otomatik refactoring araçları ile daha temiz ve bakımı kolay kod yazmaya teşvik eder.



Performans Standartları: Milisaniyeler Savaşı

E-ticaret veya finansal uygulamalarda, yanıt süresindeki her milisaniye, kullanıcı etkileşimi ve gelir üzerinde doğrudan etkiye sahiptir. ASGI, I/O bound (giriş/çıkış yoğun) operasyonlarda yüksek eşzamanlılık sağlayarak, modern API'lerden beklenen düşük gecikme süresi (low latency) hedefine ulaşmamızı sağlar.



/health endpoint'i basit görünse de, DevOps ve sistem izleme (monitoring) için kritiktir.

Bu endpoint, yük dengeleyiciler (load balancers) ve konteyner orkestrasyon araçları (Kubernetes) tarafından uygulamanın canlı ve istek almaya hazır olup olmadığını kontrol etmek için kullanılır.

Yanıtın 'ok' olması, sistem sağlığının temel bir göstergesidir.



Sanal Ortamların Derinlemesine İncelenmesi

Global Python ortamına doğrudan kurulum yapıldığında, kütüphaneler sistem çapında karışır ve sürüm çakışmaları kaçınılmaz olur. Sanal ortamlar, bağımlılıkları projenin 'venv' dizininde tutarak bu kirliliği önler. Bu, bir projenin 'requirements.txt' dosyasından kolayca yeniden oluşturulabilir olmasını sağlar (Tekrarlanabilirlik).



Uvicorn'un Mimari Rolü

FastAPI (uygulama çatısı), iş mantığını sağlar. Uvicorn ise (sunucu) HTTP isteklerini alır ve bunları FastAPI uygulamasına ileten asıl sunucu motorudur. Uvicorn, ASGI'yi desteklediği için FastAPI'nin asenkron yeteneklerini tam olarak kullanmasına izin verir.

Alternatif ASGI sunucuları (Hypercorn) da mevcuttur, ancak Uvicorn FastAPI ile en çok kullanılan ve optimize edilmiş seçenektir.

Swagger UI Alternatifi: Redoc

FastAPI, Swagger UI'a ek olarak Redoc tabanlı bir dokümantasyon da sunar. <http://127.0.0.1:8000/redoc> adresinde mevcuttur. Redoc, daha okuma odaklı, tek sayfalık ve görsel olarak daha temiz bir dokümantasyon deneyimi sunar, genellikle API'yi tüketen geliştiriciler için tercih edilir.

HTTP Metotlarının Detaylı İncelenmesi

GET: Kaynak okuma veya sorgulama (@app.get).

POST: Yeni bir kaynak oluşturma (@app.post).

PUT: Mevcut bir kaynağın tamamını güncelleme (@app.put).

DELETE: Bir kaynağı sunucudan kaldırma (@app.delete).

Bu metotlar, Veritabanı İşlemleri (CRUD: Create, Read, Update, Delete) ile doğrudan ilişkilidir.



FastAPI'de Tip İpuçlarının Kullanımı

FastAPI, Python'ın tip ipuçlarından sadece Pydantic doğrulaması için değil, aynı zamanda PyCharm'ın statik analiz araçları için de yararlanır. Tip ipuçları, kodun okunabilirliğini artırır ve IDE'nin hata potansiyelini önceden belirlemesine yardımcı olur. Bu, özellikle büyük kod tabanlarında hatalı veri türü kullanımlarını engeller.

Git'in Önemi ve Temel Komutlar

Tüm profesyonel yazılım projelerinde, kod değişikliklerini takip etmek ve geri alabilmek için Git kullanılır.

Temel Komutlar:

git init (depoyu başlatır)

git add . (değişiklikleri hazırlar)

git commit -m 'mesaj' (değişiklikleri kaydeder)

git push (uzak depoya gönderir)

PyCharm Terminal Kullanımı

PyCharm, projeye ilişkili sanal ortamın otomatik olarak yüklendiği entegre bir terminale sahiptir.

Bu terminali kullanmak, harici komut satırı pencerelerine geçiş yapma ihtiyacını ortadan kaldırır.

Sanal ortam aktifleştirme ve kütüphane kurulumları gibi işlemler bu entegre terminal üzerinden yapılmalıdır.



Geliştirme Hızı ve Basitlik

FastAPI, Python sözlüklerinden otomatik JSON yanıtı oluşturma gibi basit yaklaşımları benimseyerek boilerplate (tekrar eden kalıp) kod miktarını azaltır. Bu, geliştiricilerin çekirdek iş mantığına daha fazla odaklanmasını sağlar ve hızlı prototiplemeye olanak tanır.



Sıfırdan Proje Kurulum Adımlarının Özeti

1. Python, PyCharm ve Git Kurulumu.
2. PyCharm'da yeni proje oluşturma.
3. Sanal ortamı (venv) oluşturma ve aktifleştirme.
4. 'pip install fastapi uvicorn' komutuyla kütüphaneleri yükleme.
5. 'main.py' dosyasını oluşturup 'app = FastAPI()' nesnesini yaratma.
6. '/health' endpoint'ini tanımlama.

Sunucu Başlatma ve Test Akışının Özeti

1. Terminalde 'uvicorn main:app --reload' komutunu çalıştır.
2. Tarayıcıda 'http://127.0.0.1:8000/health' adresine giderek '{'status': 'ok'}' yanıtını kontrol et.
3. 'http://127.0.0.1:8000/docs' adresinde Swagger UI dokümantasyonunun otomatik oluştuğunu gör.

Görsel Hata Ayıklamanın Değeri

Debugging sadece hataları bulmakla kalmaz, aynı zamanda kodun çalışma mekaniğini (internals) anlamak için en iyi pedagojik araçtır. Öğrenciler, bir fonksiyon çağrısının stack'te nasıl ilerlediğini ve verinin nasıl dönüştürüldüğünü görerek daha derin bir anlayış geliştirirler. Bu, ilerideki middleware, bağımlılık enjeksiyonu ve veritabanı işlemlerinde çok kritik olacaktır.



Hafta 1: Kapanış ve Motivasyon

Bu hafta, kodlamadan çok kurulum ve motivasyona odaklandık. Önümüzdeki 7 haftalık yolculuğun sonunda, tam teşekküllü, güvenli ve mobil entegrasyon için hazır bir API'ye sahip olacaksınız. Kurulum ve debugging becerileri, tüm projenin temel taşıdır.



Ek Konu: Python Sanal Ortam Alternatifleri

venv basit Python projeleri için yeterli olsa da, büyük veri veya veri bilimi projelerinde Conda (Anaconda) tercih edilebilir.

Poetry, bağımlılık yönetimi ve paketleme süreçlerini venv'den daha gelişmiş bir şekilde ele alan modern bir araçtır.

FastAPI ve PyCharm ile çalışırken venv, öğrenci projeleri için en standart ve hafif çözümdür.



Ek Konu: Debugger'da Koşullu Breakpointler

Büyük döngüler veya sık çağrılan fonksiyonlar için her seferinde durmak verimsizdir.

Koşullu Breakpointler: Sadece belirli bir koşul (örneğin, bir değişkenin değeri 0'dan küçük olduğunda) doğru olduğunda durması için ayarlanan breakpointlerdir.

Bu, hata ayıklama süresini dramatik şekilde azaltır.

Ek Konu: Python Sözlüklerinin JSON'a Çevrimi

Python'daki sözlükler (dictionaries) ve listeler, JSON'daki nesneler ve dizilere doğal olarak eşlenir.

FastAPI, yanıt döndürülmeden önce 'jsonable_encoder' mekanizmasını kullanarak bu yerel Python nesnelerini HTTP uyumlu JSON formatına otomatik olarak dönüştürür.

Bu işlem, geliştiricinin manuel serileştirme kodu yazma ihtiyacını ortadan kaldırır.



ASGI Server Süreçleri ve Multiprocessing

Geliştirme modunda Uvicorn tek bir süreçte çalışsa da, üretimde birden fazla işçi süreci (worker process) kullanılır.

Bu worker'lar, işletim sisteminin çekirdeklerini kullanarak paralel işlem yapma yeteneği sağlar.

ASGI, bu çoklu süreçleri asenkron olarak yöneterek uygulamanın binlerce eşzamanlı bağlantıyı idare etmesini mümkün kılar.



Sürüm Yönetimi: requirements.txt

Bir sanal ortamı oluşturduktan ve tüm kütüphaneleri yükledikten sonra, bağımlılıkları kaydetmek standart bir pratiktir.

`pip freeze > requirements.txt` komutu, o anki ortamda kurulu olan tüm kütüphaneleri ve kesin sürümlerini bu dosyaya yazar.

Başka bir geliştirici, bu dosyayı kullanarak '`pip install -r requirements.txt`' ile aynı ortamı kolayca yeniden yaratabilir.

Hata Kodları ve HTTP Durumları

API'ler, istemciye işlemin sonucunu bildirmek için HTTP durum kodlarını kullanır:

- 200 OK (Başarılı).
- 404 Not Found (Kaynak bulunamadı).
- 422 Unprocessable Entity (Veri doğrulama hatası).
- 500 Internal Server Error (Sunucu tarafı hatası).

'/health' endpoint'i başarılı olduğunda 200 durum kodu döndürür.

RESTful API Tasarım Prensipleri

RESTful API'lerde odak noktası kaynaklardır (örneğin, users, items, threats). URL'ler, fiiller yerine isimleri (çoğul olarak) içermelidir: '/users' yerine '/getUsers' yanlış bir kullanımdır. Bu hafta oluşturduğumuz '/health' endpoint'i bir durum kaynağını temsil eder.

PyCharm Proje Yapılandırması

PyCharm, sanal ortamı otomatik olarak projenin Python Yorumlayıcısı (Interpreter) olarak ayarlamalıdır.

Ayarlar > Proje > Python Interpreter kısmından doğru 'venv' dizinindeki yorumlayıcının seçili olduğunu kontrol etmek önemlidir.

Yanlış yorumlayıcı seçimi, 'fastapi' kütüphanesini bulamama hatalarına yol açar.



Dekoratorların Çalışma Mekanizması

`@app.get('/health')` dekoratörü çalıştığında, FastAPI nesnesi (`app`) altında bir kayıt tutulur.

Bu kayıt, hangi URL'nin (path) hangi HTTP metoduyla (GET) eşleştiğini ve bu isteğin hangi Python fonksiyonu tarafından (`read_health`) karşılanacağını belirtir.

Bu kayıt tablosu, gelen isteklerin doğru yere yönlendirilmesini sağlar.

Uygulama Modeli: Uygulama Fabrikaları

Basit projelerde 'app = FastAPI()' global olarak tanımlanabilir. Ancak büyük projelerde, uygulamanın dinamik olarak ayarlandığı ve döndürüldüğü 'Uygulama Fabrikası' (Application Factory) desenleri kullanılır. Bu, test edilebilirlik ve yapılandırma esnekliğini artırır. FastAPI'nin Router yapısı bu modülerliği destekler.

Geliştirme Ortamı Seçimi: CLI vs IDE

PyCharm'in entegre terminali faydalı olsa da, profesyonel geliştiricilerin komut satırı (CLI) araçlarına hakim olması gerekir.

venv aktiveleştirme, pip kurulumu ve uvicorn başlatma gibi işlemlerin altında yatan mekanizmaları bilmek önemlidir.

Bu, sunucunun bulut ortamlarında (Docker, AWS) nasıl çalıştırılacağını anlamayı kolaylaştırır.



Statik Dosya Sunumu ve FastAPI

Bu hafta sadece JSON yanıtları döndürdük.

Ancak API'ler bazen resimler, CSS dosyaları veya HTML gibi statik dosyaları sunmalıdır.

FastAPI, 'StaticFiles' modülü aracılığıyla bu tür dosyaların sunumunu kolayca yönetebilir.



Gecikme Süresi (Latency) Kavramı

Gecikme süresi, bir isteğin sunucuya ulaşması ile yanıtın istemciye geri dönmesi arasında geçen süredir.

FastAPI'nin ASGI yapısı, genellikle düşük gecikme süresi sağlar.

Performans testleri (load testing), API'nin farklı yük seviyelerinde gecikme süresini nasıl koruduğunu değerlendirmek için kullanılır.



Python Sözlüklerinin Serileştirme Kısıtlamaları

Basit bir Python sözlüğü (dictionary) kullanmak '/health' endpoint'i için yeterlidir.

Ancak, daha karmaşık veri tipleri (örneğin, tarih/saat nesneleri veya özel sınıflar) için FastAPI, Pydantic modelleri kullanılmasını önerir.

Pydantic, bu nesneleri güvenli ve standart bir JSON formatına dönüştürme konusunda uzmandır.



Çevre Değişkenleri (Environment Variables)

Profesyonel projelerde, port numarası, veritabanı şifreleri veya API anahtarları gibi yapılandırma bilgileri doğrudan koda yazılmaz. Bunun yerine, çevre değişkenleri kullanılır. FastAPI projeleri, 'python-dotenv' veya Pydantic'in Ayarlar Yönetimi (Settings Management) ile bu değişkenleri güvenli bir şekilde yükleyebilir.

Debugging'de İzleme Pencereleeri

Breakpoint durduğunda, sadece 'Variables' panelindeki mevcut değişkenlere bakmak zorunda değilsiniz.

'Watch' penceresi: Program çalışırken sürekli olarak takip etmek istediğiniz karmaşık ifadeleri veya nesne özelliklerini eklemenizi sağlar.

'Evaluate Expression': Programın durduğu noktada rastgele Python kodları çalıştırarak test yapmanıza olanak tanır.

Tekrarlanabilirlik ve Reproducible Builds

Yazılım geliştirme sürecinde bir projenin tam olarak aynı şekilde (aynı kütüphane versiyonları ile) farklı makinelerde veya üretim sunucularında çalıştırılabilmesi zorunludur.

Sanal ortamlar ve 'requirements.txt' dosyası, bu tekrarlanabilirliği garanti altına alır, 'benim makinemde çalışıyordu' problemini çözer.



FastAPI'nin Açık Kaynak Topluluğu

FastAPI, modern, aktif ve hızla büyüyen bir açık kaynak topluluğuna sahiptir. Büyük bir topluluk, daha iyi dokümantasyon, daha hızlı hata düzeltmeleri ve çok sayıda üçüncü taraf entegrasyonu (veritabanları, kimlik doğrulama) anlamına gelir.

Bu, bir çatının uzun ömürlü olması için önemli bir göstergedir.



Async/Await Kullanımı

FastAPI, varsayılan olarak asenkron fonksiyonları bekler (`async def`). Eğer fonksiyonunuz I/O yoğun bir işlem yapıyorsa (veritabanı, harici API), bu fonksiyonu '`async def`' olarak tanımlamanız gerekir. Eğer fonksiyonunuz sadece CPU yoğun bir işlem yapıyorsa, FastAPI onu ayrı bir iş parçacığında (thread) çalıştırarak ana döngüyü engellemesini önler.

Gelecek Haftanın Konuları

Önümüzdeki hafta, dinamiğe geçiyoruz: Path Parametreleri ve Query Parametreleri.

Kullanıcıdan veri almayı ve bu verilere göre API'mizin yanıt vermesini öğreneceğiz.

Bu, '/items/5' gibi dinamik URL'lerin nasıl tanımlandığını içerecektir.

Bu hafta öğrendiklerimiz:
API'lerin modern webdeki konumu.
FastAPI ve PyCharm kurulumu.
Sanal ortamların kritikliği.
İlk '/health' endpoint'inin oluşturulması.
PyCharm debugger ile kod akışını izleme.

Uygulama Programlama Arayüzleri (API) ve Modern Web Mimarisi

Süleyman **EZDEMİR**

suleyman.ezdemir@giresun.edu.tr

